

```
#!/usr/bin/env python3
```

```
"""
```

Escáner de puntos de quiebre (breakouts)

Qué detecta (por cada símbolo de tu lista):

- RUPTURA ALCISTA: el cierre supera el máximo de las últimas N velas y el volumen es mayor que VOL\_MULT x el volumen promedio.
- RUPTURA BAJISTA: el cierre rompe el mínimo de las últimas N velas con el mismo filtro de volumen.
- Calcula un stop sugerido (ATR) y un objetivo 2R como referencia.

Instalación:

```
pip install yfinance pandas requests
```

Uso:

```
python breakout_scanner.py          # escanea una vez
python breakout_scanner.py --loop    # escanea cada CHECK_EVERY segundos
python breakout_scanner.py --symbols AAPL,MSFT,BTC-USD --interval 1h
```

Alertas por Telegram (opcional):

```
export TELEGRAM_TOKEN="tu_token_del_bot"
export TELEGRAM_CHAT_ID="tu_chat_id"
```

Aviso: es una herramienta de apoyo, no una recomendación de inversión.

Las rupturas fallan seguido; usa siempre gestión de riesgo.

```
"""
```

```
import argparse
```

```
import os
```

```
import sys
```

```
import time
```

```
from datetime import datetime
```

```
import pandas as pd
```

```
import requests
```

```
import yfinance as yf
```

```
# ----- CONFIGURACIÓN -----
```

```
WATCHLIST = [
```

```
    "AAPL", "MSFT", "NVDA", "TSLA", "AMZN", "META", "GOOGL",    # acciones
```

```
    "SPY", "QQQ",                                                # ETFs
```

```
    "BTC-USD", "ETH-USD",                                         # cripto
```

```
    "EURUSD=X",                                                  # forex
```

```
]
```

```

INTERVAL = "1d"          # 1m, 5m, 15m, 30m, 1h, 1d, 1wk
PERIOD = "1y"            # historial a descargar (en intradía yfinance limita: 5m/1
LOOKBACK = 20            # velas para definir resistencia/soporte
VOL_MULT = 1.5           # el volumen debe superar X veces el promedio
ATR_N = 14               # periodo del ATR
ATR_STOP = 1.5           # stop = cierre +/- ATR_STOP * ATR
RR = 2.0                 # objetivo = RR veces el riesgo
CHECK_EVERY = 300        # segundos entre escaneos en modo --loop
# -----

```

```

alertas_enviadas = set() # evita repetir la misma alerta

```

```

def atr(df: pd.DataFrame, n: int) -> pd.Series:

```

```

    prev_close = df["Close"].shift(1)
    tr = pd.concat(
        [
            df["High"] - df["Low"],
            (df["High"] - prev_close).abs(),
            (df["Low"] - prev_close).abs(),
        ],
        axis=1,
    ).max(axis=1)
    return tr.rolling(n).mean()

```

```

def analizar(symbol: str, interval: str, period: str):

```

```

    df = yf.Ticker(symbol).history(period=period, interval=interval, auto_adjust=
    df = df.dropna(subset=["High", "Low", "Close"])
    if len(df) < max(LOOKBACK, ATR_N) + 2:
        return None

```

```

    # Niveles calculados SOLO con velas anteriores a la última
    resistencia = df["High"].shift(1).rolling(LOOKBACK).max()
    soporte = df["Low"].shift(1).rolling(LOOKBACK).min()
    vol_prom = df["Volume"].shift(1).rolling(LOOKBACK).mean()
    df["ATR"] = atr(df, ATR_N)

```

```

    ultima = df.iloc[-1]
    ts = df.index[-1]
    res, sop, vp = resistencia.iloc[-1], soporte.iloc[-1], vol_prom.iloc[-1]
    cierre, vol, a = ultima["Close"], ultima["Volume"], ultima["ATR"]

```

```

    # En forex/algunos índices el volumen viene en 0: no se exige confirmación
    vol_ok = True if (pd.isna(vp) or vp == 0) else vol >= VOL_MULT * vp

```

```

    if cierre > res and vol_ok:

```

```

        stop = cierre - ATR_STOP * a
        objetivo = cierre + RR * (cierre - stop)
        return dict(symbol=symbol, tipo="ALCISTA", ts=ts, cierre=cierre,
                    nivel=res, stop=stop, objetivo=objetoivo,
                    vol_ratio=(vol / vp if vp else float("nan")))
    if cierre < sop and vol_ok:
        stop = cierre + ATR_STOP * a
        objetivo = cierre - RR * (stop - cierre)
        return dict(symbol=symbol, tipo="BAJISTA", ts=ts, cierre=cierre,
                    nivel=sop, stop=stop, objetivo=objetoivo,
                    vol_ratio=(vol / vp if vp else float("nan")))
    return None

def formatear(s: dict) -> str:
    emoji = "🟢" if s["tipo"] == "ALCISTA" else "🔴"
    return (
        f"{emoji} RUPTURA {s['tipo']} - {s['symbol']}\n"
        f"Cierre: {s['cierre']:.4f} (nivel roto: {s['nivel']:.4f})\n"
        f"Volumen: {s['vol_ratio']:.1f}x el promedio\n"
        f"Stop sugerido: {s['stop']:.4f} | Objetivo {RR:.0f}R: {s['objetivo']:.4f}
        f"Vela: {s['ts']}"
    )

def notificar(msg: str):
    print("\n" + msg + "\a") # \a = sonido de la terminal
    token = os.getenv("TELEGRAM_TOKEN")
    chat_id = os.getenv("TELEGRAM_CHAT_ID")
    if token and chat_id:
        try:
            requests.post(
                f"https://api.telegram.org/bot{token}/sendMessage",
                data={"chat_id": chat_id, "text": msg},
                timeout=10,
            )
        except requests.RequestException as e:
            print(f"[!] No se pudo enviar a Telegram: {e}")

def escanear(symbols, interval, period):
    print(f"[{datetime.now():%Y-%m-%d %H:%M:%S}] Escaneando {len(symbols)} símbolo
    encontrados = 0
    for sym in symbols:
        try:
            senal = analizar(sym, interval, period)
        except Exception as e:

```

```

        print(f"[!] {sym}: {e}")
        continue
    if not senal:
        continue
    clave = (senal["symbol"], str(senal["ts"]), senal["tipo"])
    if clave in alertas_enviadas:
        continue
    alertas_enviadas.add(clave)
    notificar(formatear(senal))
    encontrados += 1
if encontrados == 0:
    print("Sin rupturas nuevas.")

def main():
    global LOOKBACK, VOL_MULT
    p = argparse.ArgumentParser(description="Escáner de rupturas")
    p.add_argument("--symbols", help="Lista separada por comas (ej: AAPL,BTC-USD)")
    p.add_argument("--interval", default=INTERVAL)
    p.add_argument("--period", default=PERIOD)
    p.add_argument("--lookback", type=int, default=LOOKBACK)
    p.add_argument("--vol", type=float, default=VOL_MULT)
    p.add_argument("--loop", action="store_true", help="Repetir cada CHECK_EVERY")
    args = p.parse_args()

    LOOKBACK, VOL_MULT = args.lookback, args.vol
    symbols = [s.strip().upper() for s in args.symbols.split(",")] if args.symbol

    if not args.loop:
        escanear(symbols, args.interval, args.period)
        return
    try:
        while True:
            escanear(symbols, args.interval, args.period)
            time.sleep(CHECK_EVERY)
    except KeyboardInterrupt:
        print("\nDetenido.")
        sys.exit(0)

if __name__ == "__main__":
    main()

```